

基本内容

常见的搜索树结构：
k叉树(k皇后问题)
子集树(0-1背包问题)
排列树(货郎问题)

基本思想

- 适用问题：求解搜索问题
- 搜索空间：树，结点对应部分解向量，树叶对应可行解
- 搜索过程：采用系统的方法隐含遍历搜索树
- 搜索策略：深度优先，宽度优先，函数优先，宽深结合等
- 结点分支判定条件：
 - 满足约束条件——分支扩张解向量
 - 不满足约束条件，回溯到该结点的父结点
- 结点状态：动态生成
 - 白结点(尚未访问)；
 - 灰结点(正在访问该结点为根的子树)；
 - 黑结点(该结点为根的子树遍历完成)
- 存储：当前路径

必要条件：多米诺性质

设 $P(x_1, x_2, \dots, x_i)$ 为真表示向量 $\langle x_1, x_2, \dots, x_i \rangle$ 中 i 个皇后放置在彼此不能攻击的位置

$$P(x_1, x_2, \dots, x_{k+1}) \rightarrow P(x_1, x_2, \dots, x_k) \quad 0 < k < n$$

例4 求不等式的整数解 $7P(x_1, x_2, \dots, x_k) \Rightarrow 7P(x_1, x_2, \dots, x_{k+1})$
 $5x_1 + 4x_2 - x_3 \leq 10, \quad 1 \leq x_i \leq 3, \quad i=1,2,3$ 可以正确地提前终止。

$P(x_1, \dots, x_k)$ ：意味将 $x_1, x_2, \dots, x_k$ 代入原不等式的相应部分使得左边小于等于10

不满足多米诺性质

变换：令 $x_3 = 3 - x_3'$,

$$5x_1 + 4x_2 + x_3' \leq 13, \quad 1 \leq x_1, x_2 \leq 3, \quad 0 \leq x_3' \leq 2$$

算法设计步骤

- 定义搜索问题的解向量和每个分量的取值范围
 - 解向量为 $\langle x_1, x_2, \dots, x_n \rangle$
 - 确定 x_i 的可能取值的集合为 $X_i, i=1, 2, \dots, n$.
- 当 $x_1, x_2, \dots, x_{k-1}$ 确定以后计算 x_k 取值集合 $S_k, S_k \subseteq X_k$
- 确定结点儿子的排列规则
- 判断是否满足多米诺性质
- 搜索策略——深度优先、宽度优先等
- 确定每个结点分支约束条件
- 确定存储搜索路径的数据结构

估计时间复杂度 —— 结点数 $\times$ 每结点时间

Monte Carlo方法

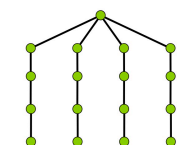
1. 从根开始，随机选择一条路径，直到不能分支为止，即从 $x_1, x_2, \dots$ 依次对 x_i 赋值，每个 x_i 的值是从当时的 S_i 中随机选取，直到向量不能扩张为止。
2. 假定搜索树的其他 $|S_i|-1$ 个分支与以上随机选出的路径一样，计数搜索树的点数。
3. 重复步骤1和2，将结点数进行概率平均。

例5 估计四后问题的效率

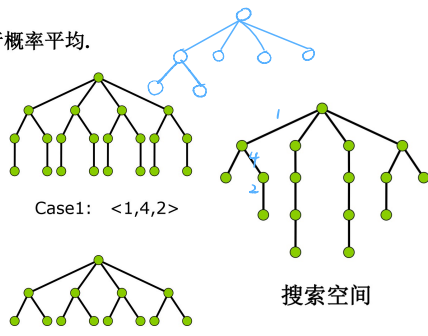
case1. $\langle 1, 4, 2 \rangle$: $1+4+4 \times 2+4 \times 2=21$

case2. $\langle 2, 4, 1, 3 \rangle$: $4 \times 4+1=17$

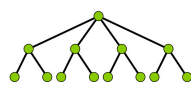
case3. $\langle 1, 3 \rangle$: $1+4 \times 1+4 \times 2=13$



Case2: $\langle 2, 4, 1, 3 \rangle$



Case1: $\langle 1, 4, 2 \rangle$



Case3: $\langle 1, 3 \rangle$

假设4次抽样测试：

case1:1次，case2:1次，case3:2次，

平均结点数 $= (21 \times 1 + 17 \times 1 + 13 \times 2) / 4 = 16$

搜索空间访问的结点数为17

Monte Carlo

1. $sum \leftarrow 0$ //sum为t次结点平均数
2. for $i \leftarrow 1$ to t do //取样次数t
3. $m \leftarrow \text{Estimate}(n)$ //m为本次结点总数
4. $sum \leftarrow sum + m$
5. $sum \leftarrow sum / t$

*m为输出——本次取样结点总数，k为层数， r_1 为本层分支数， r_2 为上层分支数，n为树的层数

算法Estimate(n)

1. $m \leftarrow 1; r_2 \leftarrow 1; k \leftarrow 1$ //m为结点总数
2. While $k \leq n$ do
3. if $S_k = \emptyset$ then return m
4. $r_1 \leftarrow |S_k| * r_2$ //r1为扩张后结点总数
5. $m \leftarrow m + r_1$ //r2为扩张前结点总数
6. $x_k \leftarrow$ 随机选择 S_k 的元素
7. $r_2 \leftarrow r_1$
8. $k \leftarrow k+1$

扩张-选择。

影响算法效率的因素

最坏情况下的时间 $W(n) = (p(n)f(n))$

其中 $p(n)$ 为每个结点时间， $f(n)$ 为结点个数

影响回溯算法效率的因素

搜索树的结构

分支情况：分支均匀否

树的深度

对称程度：对称适合裁减

解的分布

在不同子树中分布多少是否均匀

分布深度

△约束条件的判断：计算简单

算法改进途径

- 根据树分支设计优先策略：
 - 结点少的分支优先，解多的分支优先
- 利用搜索树的对称性剪裁子树
- 分解为子问题：
 - 求解时间 $f(n) = c2^n$ ，组合时间 $T = O(f(n))$
 - 如果分解为k个子问题，每个子问题大小为 n/k
 - 求解时间为

$$\frac{n}{k} c 2^{n/k} + T$$

分支限界技术

- 设立代价值函数 每个结点，对应一个代价函数
 - 函数值以该结点为根的搜索树中的所有可行解的目标函数值的上界 → 针对最大化
 - 父结点的代价值不小于子结点的代价值
- 设立界
 - 代表当时已经得到的可行解的目标函数的最大值
 - 界的设定初值可以设为0
 - 可行解的目标函数值大于当时的界，进行更新
- 搜索中停止分支的依据
 - 不满足约束条件或者其代价值函数小于当时的界

典型例题

背包问题

背包问题的实例：

$$\begin{aligned} \max & x_1 + 3x_2 + 5x_3 + 9x_4 \\ 2x_1 + 3x_2 + 4x_3 + 7x_4 & \leq 10 \\ x_i \in \mathbb{N}, i &= 1, 2, 3, 4 \end{aligned}$$

对变元重新排序使得

$$\frac{v_i}{w_i} \geq \frac{v_{i+1}}{w_{i+1}}$$

排序后实例

$$\begin{aligned} \max & 9x_1 + 5x_2 + 3x_3 + x_4 \\ 7x_1 + 4x_2 + 3x_3 + 2x_4 & \leq 10 \\ x_i \in \mathbb{N}, i &= 1, 2, 3, 4 \end{aligned}$$

结点 $\langle x_1, x_2, \dots, x_k \rangle$ 的代价函数：

$$f(\langle x_1, x_2, \dots, x_k \rangle) = \begin{cases} \sum_{i=1}^k v_i x_i + (b - \sum_{i=1}^k w_i x_i) \frac{v_{k+1}}{w_{k+1}}, & \exists j > k, w_j \leq \sum_{i=1}^k w_i x_i \\ \sum_{i=1}^k v_i x_i, & \text{否则} \end{cases}$$

搜索策略：广度优先

最大团问题

问题：给定无向图 $G = \langle V, E \rangle$, 求 G 中的最大团。

相关知识：

- 无向图 $G = \langle V, E \rangle$,
- G 的子图: $G' = \langle V', E' \rangle$, 其中 $V' \subseteq V, E' \subseteq E$,
- G 的补图: $\bar{G} = \langle V, E' \rangle$, E' 是 E 关于完全图边集的补集
- G 中的团: G 的完全子图
- G 的点独立集: G 的顶点子集 A , 且 $\forall u, v \in A, (u, v) \notin E$.
- 最大团: 顶点数最多的团
- 最大点独立集: 顶点数最多的点独立集

命题: U 是 G 的最大团当且仅当 U 是 $\bar{G}$ 的最大点独立集

解向量: $\langle x_1, x_2, \dots, x_n \rangle, x_i \in \{0, 1\}, i = 1, 2, \dots, n$

$\langle x_1, x_2, \dots, x_k \rangle$ 表示已搜索 k 个结点, 其中 $x_i = 1$ 的结点在

当前团内

搜索树: 子集树

约束条件: 顶点与当前团内每个顶点均有边相连

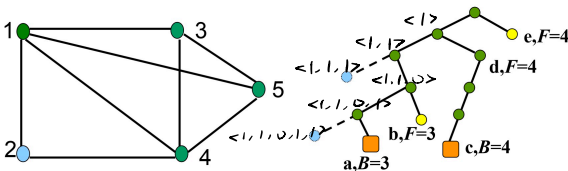
界: 当前团中已搜索到的极大团的顶点数

代价函数: $F = C_n + n - k, C_n$ 为当前团的顶点数 (初始为 0)

k 为结点层数。

时间: $O(2^n \cdot n)$

搜索策略: 广度优先



- a: 得第一个极大团 $\{1, 2, 4\}$, 顶点数为3, 界为3;
 - b: 代价函数值 $F=3$, 回溯;
 - c: 得第二个极大团 $\{1, 3, 4, 5\}$, 顶点数为4, 修改界为4;
 - d: 不必搜索其它分支, 因为 $F=4$, 不超过界;
 - e: $F=4$, 不必搜索。
- 最大团为 $\{1, 3, 4, 5\}$, 顶点数为 4。

货郎问题

问题: 给定 n 个城市集合 $C = \{c_1, c_2, \dots, c_n\}$, 从一个城市到另一个城市的距离 d_{ij} 为正整数, 求一条最短且每个城市恰好经过一次的巡回路线。

货郎问题的类型: 有向图、无向图。

设巡回路线从 1 开始,

解向量为 $\langle i_1, i_2, \dots, i_{n-1} \rangle$,

其中 $i_1, i_2, \dots, i_{n-1}$ 为 $\{2, 3, \dots, n\}$ 的排列。

搜索空间为排列树, 结点 $\langle i_1, i_2, \dots, i_k \rangle$ 表示得到 k 步路线

约束条件: 令 $B = \{i_1, i_2, \dots, i_k\}$, 则

$$i_{k+1} \in \{2, \dots, n\} - B$$

界: 当前得到的最短巡回路线长度

代价函数: 设顶点 c_i 出发的最短边长度为 l_i , d_j 为选定巡回路线中第 j 段的长度, 则

$$L = \sum_{j=1}^k d_j + l_{i_k} + \sum_{i_j \notin B} l_{i_j} \quad L(\langle i_1, i_2, \dots, i_k \rangle)$$

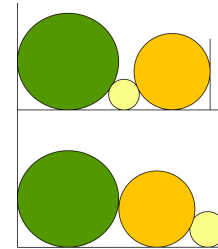
为部分巡回路线扩张成全程巡回路线的长度下界

时间 $O(n!)$: 计算 $O((n-1)!)$ 次, 代价函数计算 $O(n)$

$$O(n \cdot (n-1)!)$$

圆排列问题

问题: 给定 n 个圆的半径序列, 将各圆与矩形底边相切排列, 求具有最小长度 l_n 的圆的排列顺序。



解为 $\langle i_1, i_2, \dots, i_n \rangle$ 为 $1, 2, \dots, n$ 的排列, 解空间为排列树。

部分解向量 $\langle i_1, i_2, \dots, i_k \rangle$: 表示前 k 个圆已排好。令 $B = \{i_1, i_2, \dots, i_k\}$, 下一个圆选择 i_{k+1} 。

约束条件: $i_{k+1} \in \{1, 2, \dots, n\} - B$

界: 当前得到的最小圆排列长度

代价函数符号说明

k : 算法完成第 k 步, 已经选择了第 $1-k$ 个圆

r_k : 第 k 个圆的半径

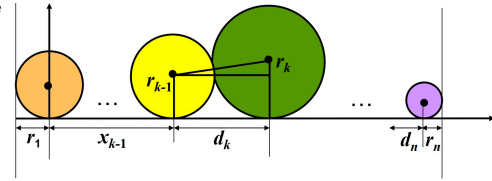
d_k : 第 $k-1$ 个圆到第 k 个圆的圆心水平距离, $k > 1$

x_k : 第 k 个圆的圆心坐标, 规定 $x_1 = 0$,

l_k : 第 $1-k$ 个圆的排列长度

L_k : 放好 $1-k$ 个圆以后, 对应结点的代价函数值

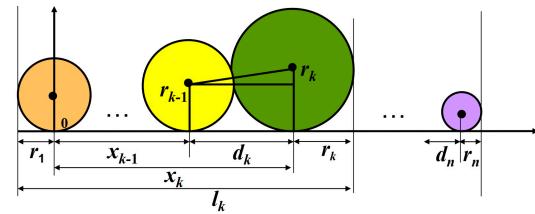
$$L_k \leq l_n$$



$$d_k = \sqrt{(r_{k-1} + r_k)^2 - (r_{k-1} - r_k)^2} = 2\sqrt{r_{k-1}r_k}$$

$$x_k = x_{k-1} + d_k, \quad l_k = x_k + r_k + r_1$$

$$\begin{aligned} l_n &= x_k + d_{k+1} + d_{k+2} + \dots + d_n + r_n + r_1 \\ &= x_k + 2\sqrt{r_k r_{k+1}} + 2\sqrt{r_{k+1} r_{k+2}} + \dots + 2\sqrt{r_{n-1} r_n} + r_n + r_1 \end{aligned}$$



排列长度是 l_n , L 是代价函数:

$$\begin{aligned} l_n &= x_k + 2\sqrt{r_k r_{k+1}} + 2\sqrt{r_{k+1} r_{k+2}} + \dots + 2\sqrt{r_{n-1} r_n} + r_n + r_1 \\ &\geq x_k + 2(n-k)r + r + r_1 \\ L &= x_k + (2n-2k+1)r + r_1 \\ r &= \min(r_{i_j}, r_k) \quad i_j \in \{1, 2, \dots, n\} - B \end{aligned}$$

$$B = \{i_1, i_2, \dots, i_k\},$$

时间: $O(n n!) = O((n+1)!)$

连续邮资问题

问题：给定 n 种不同面值的邮票，每个信封至多 m 张，试给出邮票的最佳设计，使得从1开始，增量为1的连续邮资区间达到最大？

实例： $n=5, m=4$,
面值 $X_1=<1,3,11,15,32>$ ，邮资连续区间为 $\{1, 2, \dots, 70\}$
面值 $X_2=<1,6,10,20,30>$ ，邮资连续区间为 $\{1, 2, 3, 4\}$

可行解： $\langle x_1, x_2, \dots, x_n \rangle, x_1=1, x_1 < x_2 < \dots < x_n$
约束条件：在结点 $\langle x_1, x_2, \dots, x_i \rangle$ 处，邮资最大连续区间为 $\{1, \dots, r_i\}$ ， x_{i+1} 的取值范围是 $\{x_i+1, \dots, r_i+1\}$

$y_i(j)$ ：用至多 m 张面值 x_i 的邮票加上 $x_1, x_2, \dots, x_{i-1}$ 面值的邮票贴 j 邮资时的最少邮票数，则

$$y_i(j) = \min_{1 \leq t \leq m} \{t + y_{i-1}(j - t x_i)\}$$

$$y_1(j) = j$$

$$r_i = \min\{j \mid y_i(j) \leq m, y_i(j+1) > m\}$$

搜索策略：深度优先
界： max, m 张邮票可付的连续区间的最大邮资

- ①解向量 + 含义
- ②搜索空间 + 策略
- ③约束条件
- ④界
- ⑤代价函数
- ⑥时间.